



Test Project

17 Web Technologies

Module A Mini Speed Test Project



**BAGONG
PILIPINAS**



**Sa TESDA,
kayangKaya**



Contents

Introduction 3
Description of project and tasks 3
Tasks for Design Implementation 3
Tasks for Front-end Development..... 5
Tasks for Back-end Development 8
Instructions to the Competitor 10
Other 10

Thailand (CE)	Cambodia (DCE)	Indonesia (E)	Malaysia (E)	Philippines (E)
Philippines (E)	Singapore (E)	Thailand (E)		

Introduction

In this mini speed Test Project, you are given multiple mini tasks. We have collected all the tasks from multiple clients. And for priority reason, we have selected some tasks for this round of development. Please only implement the required tasks.

Please follow the instruction to know which one to implement.

For each mini Test Project, there are three levels.

- Easy: 0.5 points worth that is expected takes less than 15 minutes of work.
- Medium: 1.0 points worth that is expected takes less than 30 minutes of work.
- Difficult: 1.5 points worth that is expected takes around 30 minutes of work.

Please put your files in /XX_module_a/ and with the task ID as folder name, e.g.

/XX_module_a/A1/ The XX is the workstation number.

Description of project and tasks

Please create an index page to link to each mini speed Test Project.

The index page should contain thumbnail and title of each mini speed test project.

Please write at least one git commit for each mini speed Test Project. The git commit messages should be readable and easy to understand.

Thailand (CE)	Cambodia (DCE)	Indonesia (E)	Malaysia (E)	Philippines (E)
Philippines (E)	Singapore (E)	Thailand (E)		

Tasks for Design Implementation

A1: Focus Frame (Easy)

Create a responsive image card that reveals four animated corner brackets when the user hovers over the card or moves keyboard focus into it. The image itself must subtly scale while remaining clipped inside the card.



Given: index.html, style.css, and media/photo.jpg, focus_frame_effect.mp4.

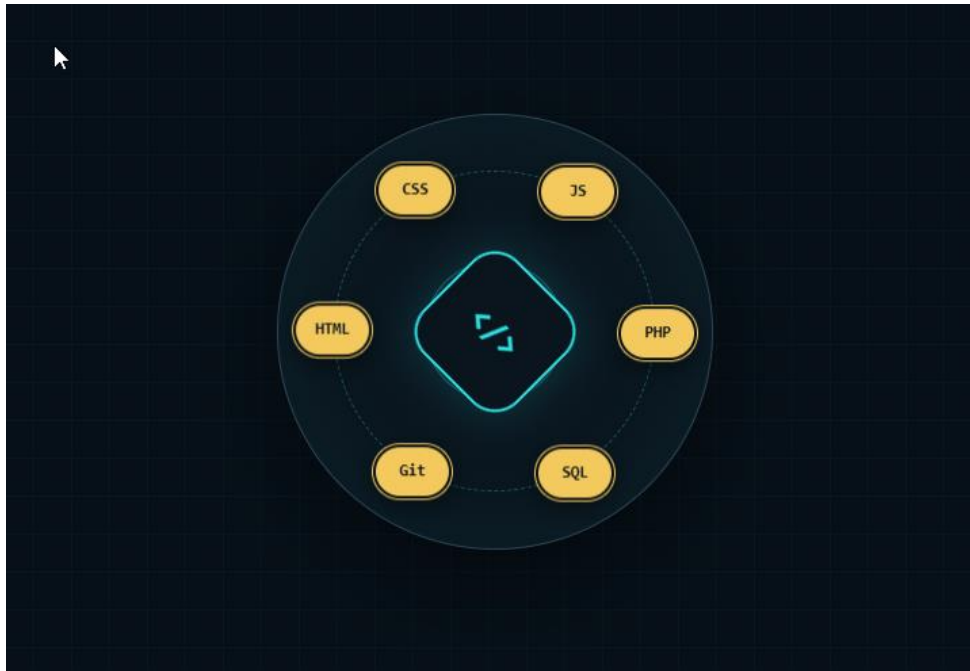
Restriction: Modify only style.css. Use HTML and CSS only.

Visual specification: Card size 360 × 240 px on wide screens; maximum width 90vw; 12 px corner radius; 3 px cyan brackets; 18 px bracket arms; 250 ms transition.

Thailand (CE)	Cambodia (DCE)	Indonesia (E)	Malaysia (E)	Philippines (E)
Philippines (E)	Singapore (E)	Thailand (E)		

A2: Orbiting Skill Badge (Medium)

Build a 320 × 320 px badge containing a central code symbol and six labeled skill nodes arranged on a circular orbit. The orbit rotates continuously while every label remains upright and readable.



Given: index.html, style.css, and orbiting_skill_badge.mp4.

Restriction: HTML and CSS only. No SVG, canvas, JavaScript, images, or external libraries.

Motion: One clockwise revolution every 12 seconds using linear infinite animation. Each node must counter-rotate by the same amount.

Acceptance checks

- All six nodes are distributed evenly around the circular path.
- The full component stays centered and scales down on viewports narrower than 360 px.
- Labels remain upright throughout the animation.
- Pausing or hovering over the badge pauses both the orbit and counter-rotation.

Thailand (CE)	Cambodia (DCE)	Indonesia (E)	Malaysia (E)	Philippines (E)
Philippines (E)	Singapore (E)	Thailand (E)		

A3: Isometric Cube in Pure CSS (Difficult)

Reproduce a three-faced isometric cube using one semantic container and three child elements. The cube must appear to have equal top, left, and right faces, with a centered `</>` mark on the front-facing pair.



Given: index.html, style.css, and isometric_cube.mp4.

Restriction: HTML and CSS only. Do not use SVG, canvas, clip-path polygon(), CSS masking, border-image, or raster images.

Dimensions: Overall visible artwork approximately 360 × 420 px on desktop, with a 1:1 face edge length of 180 px. It must shrink proportionally below 440 px viewport width.

Colors: Top #F4C95D; left #1E5A84; right #17324D; mark #FFFFFF.

Interaction: On hover or keyboard focus, lift the cube 16 px and rotate the complete object 6 degrees clockwise over 300 ms.

Acceptance checks

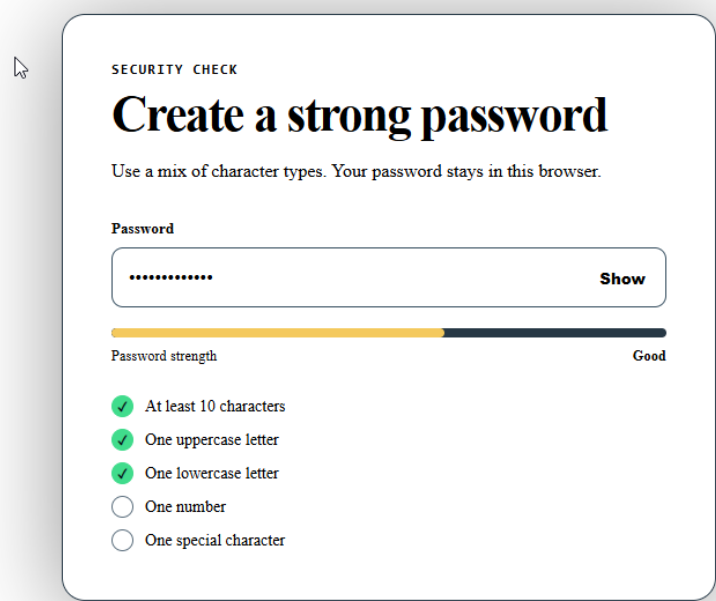
- The three faces meet without visible gaps or unintended overlap.
- Face edges have equal apparent length and the cube is centered in both axes.
- The code mark remains visually centered and readable.
- The artwork is responsive and the interactive state does not change its layout footprint.

Thailand (CE)	Cambodia (DCE)	Indonesia (E)	Malaysia (E)	Philippines (E)
Philippines (E)	Singapore (E)	Thailand (E)		

Tasks for Front-end Development

B1: Password Strength Meter (Easy)

Create a **client-side** password strength meter that updates while the user types. The starter HTML contains a password input, a meter track, a text label, and an unordered requirements list.



Given: index.html, style.css, and blank script.js

Scoring rules: Award one point for each condition: at least 10 characters; contains an uppercase letter; contains a lowercase letter; contains a digit; contains a non-alphanumeric character.

Labels: 0–1 Weak, 2 Fair, 3 Good, 4 Strong, 5 Excellent.

Acceptance checks

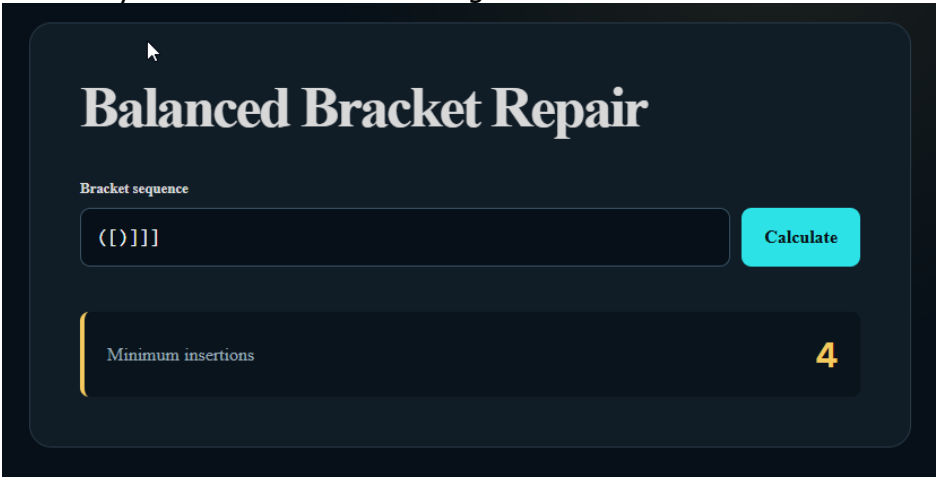
- The meter width, color class, and text label update on every input event.
- Each requirement visibly changes between unmet and met states.

The implementation works without form submission and uses no external library.

Thailand (CE)	Cambodia (DCE)	Indonesia (E)	Malaysia (E)	Philippines (E)
Philippines (E)	Singapore (E)	Thailand (E)		

B2: Balanced Bracket Repair (Medium)

Implement a JavaScript function `minimumInsertions(input)` that returns the minimum number of bracket characters required to make the input balanced. The string contains only `()`, `[]`, and `{}`. Existing characters may not be deleted or rearranged.



Given: `index.html`, `style.css`, and `blank solution.js`

Input size: 1 to 100,000 characters.

Required complexity: $O(n)$ time. The solution must not enumerate candidate strings.

```
minimumInsertions("([])") // 0
minimumInsertions("([)]") // 2
minimumInsertions("{[()]") // 3
minimumInsertions("())")  // 1
```

CLARIFICATION A closing bracket that does not match the latest unmatched opening bracket requires one insertion to close that opening bracket; processing then continues with the same closing bracket.

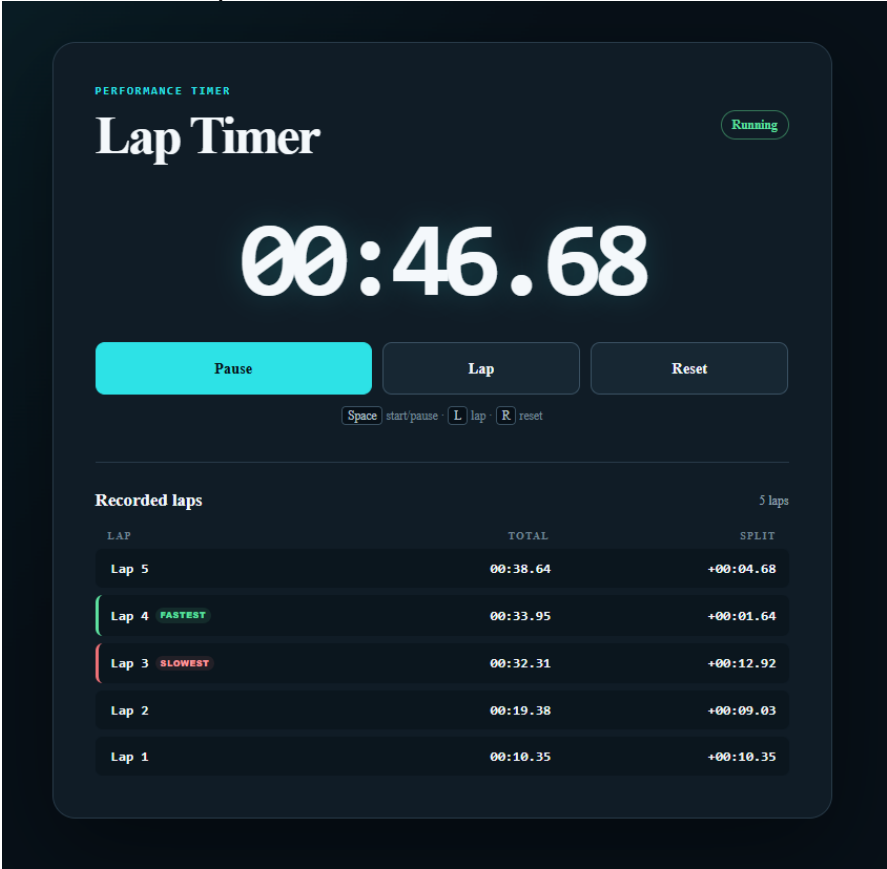
Acceptance checks

- All four sample cases return the expected values.
- The empty string returns 0 when tested by the assessor.

Thailand (CE)	Cambodia (DCE)	Indonesia (E)	Malaysia (E)	Philippines (E)
Philippines (E)	Singapore (E)	Thailand (E)		

B3: Accessible Lap Timer (Difficult)

Create a stopwatch that displays MM:SS.hh, where hh is hundredths of a second. It must support starting, pausing, resuming, recording laps, and resetting. Timing must remain accurate even when rendering frames are delayed.



Given: index.html, style.css, blank timer.js, and timer.mp4

Controls: Start/Pause toggle, Lap, and Reset. Keyboard shortcuts: Space toggles timing, L records a lap, and R resets when focus is not inside an editable field.

Lap output: Show newest lap first. Each row contains lap number, total elapsed time, and difference from the previous lap. Highlight the fastest and slowest completed laps after at least two laps.

Accuracy requirement: Use elapsed timestamps (for example, performance.now()) as the source of truth; do not calculate time by counting setInterval callbacks.

Thailand (CE)	Cambodia (DCE)	Indonesia (E)	Malaysia (E)	Philippines (E)
Philippines (E)	Singapore (E)	Thailand (E)		

Acceptance checks

- Pause and resume preserve elapsed time without adding the paused duration.
- Lap cannot be recorded while stopped or paused; Reset restores the initial state.

Fastest and slowest highlighting updates correctly when new laps are added.

Thailand (CE)	Cambodia (DCE)	Indonesia (E)	Malaysia (E)	Philippines (E)
Philippines (E)	Singapore (E)	Thailand (E)		

Tasks for Back-end Development

C1: Word Frequency Report (Easy)

Create a server-rendered form with a textarea. On submission, analyze the text case-insensitively and display the total word count, unique word count, and the five most frequent words.

Word rule: A word is a sequence of letters A–Z or a–z. Apostrophes, numbers, punctuation, and whitespace are separators.

Sorting: Sort by frequency descending, then alphabetically ascending for ties.

```
Input: Code, test, code! Build-test BUILD.  
Output: total 6; unique 3; code 2, build 2, test 2
```

Acceptance checks

- All counts are computed on the server after form submission.
- The five-word report follows the specified sorting rule.
- The original submitted text is safely redisplayed without executing markup.

C2: Palette API (Medium)

Create a server-side JSON endpoint that receives a base color and returns five deterministic shades plus a recommended black or white text color for each shade.

Request: GET /palette?color=%23336699. Accept only a six-digit hexadecimal color with an optional leading #.

Shade factors: Apply RGB channel factors 0.50, 0.75, 1.00, 1.25, and 1.50; round to the nearest integer and clamp each channel to 0–255.

Text color:

Compute relative luminance using the sRGB WCAG formula.

For each channel (R, G, B) ranging from 0 – 1 (divide 8-bit values by 255):

- If channel ≤ 0.04045 : $\text{linear_channel} = \text{channel} / 12.92$
- If channel > 0.04045 : $\text{linear_channel} = ((\text{channel} + 0.55) / 1.055) ** 2.4$

$\text{Luminance} = 0.2126 * \text{linear_channel_red} + 0.7152 * \text{linear_channel_green} + 0.0722 * \text{linear_channel_blue}$

Return #000000 when its contrast ratio is greater than or equal to white's; otherwise return #FFFFFF.

Thailand (CE)	Cambodia (DCE)	Indonesia (E)	Malaysia (E)	Philippines (E)
Philippines (E)	Singapore (E)	Thailand (E)		

```
{
  "source": "#336699",
  "shades": [
    {"factor": 0.5, "color": "#1A334D", "text": "#FFFFFF"}
  ]
}
```

Acceptance checks

- Successful responses use HTTP 200 and Content-Type application/json.
- Exactly five shades are returned in ascending factor order.
- Invalid or missing input returns HTTP 422 with a JSON error message.
- The endpoint escapes no JSON manually and exposes no warnings or stack traces.

C3: Transaction Product Import (Difficult)

Complete a CSV import service that creates or updates products without leaving the catalogue partially changed when any row is invalid.

Starter materials: A products table, database bootstrap, upload form, sample CSV, and test runner.
Complete the importer service.

Requirements

- Accept a UTF-8 CSV whose exact header is sku,name,price,stock. Process at most 500 data rows with fgetcsv.
- Trim every field. SKU must match A-Z, 0-9, and hyphen, with 3-20 characters. Name is 2-80 characters.
- Price must be a non-negative decimal with at most two fractional digits; convert it to integer cents. Stock must be a non-negative integer.
- Reject duplicate SKUs within the same file. Report the one-based CSV line and a useful validation message.
- Validate the complete file before writing. Then create new products or update existing products inside one transaction using prepared statements.
- Return a summary containing created, updated, and processed counts. Escape every value rendered in HTML.

Acceptance checks

- A valid mixed file creates new rows, updates existing rows, and reports exact counts.
- An invalid header, malformed row, invalid value, oversized file, or duplicate file SKU produces no database changes.
- Prices such as 19.90 become 1990 cents without floating-point rounding drift.
- CSV values containing commas and quotes are parsed correctly.

Thailand (CE)	Cambodia (DCE)	Indonesia (E)	Malaysia (E)	Philippines (E)
Philippines (E)	Singapore (E)	Thailand (E)		

- SQL metacharacters remain data and cannot alter the query.
- The supplied valid, rollback, and duplicate-SKU tests pass.

COMPETENCY FOCUS Structured-data parsing, defensive validation, prepared statements, integer money representation, transactions, rollback, reusable service code, and automated tests.

Instructions to the Competitor

Each mini Test Project folder should be separated and self-contained.

Please include only exactly the required mini test project. And do not include any not related folders or not related mini Test Projects.

Other

This project will be assessed by using Google Chrome web browser.

Marking Summary

	Sub-Criteria	Marks
1	Mini Test Project General	1
2	Design Implementation	3
3	Front-end Development	3
4	Back-end Development	3

Thailand (CE)	Cambodia (DCE)	Indonesia (E)	Malaysia (E)	Philippines (E)
Philippines (E)	Singapore (E)	Thailand (E)		